

Working with Instances: An Amazon EC2 Walkthrough

Learning Objectives

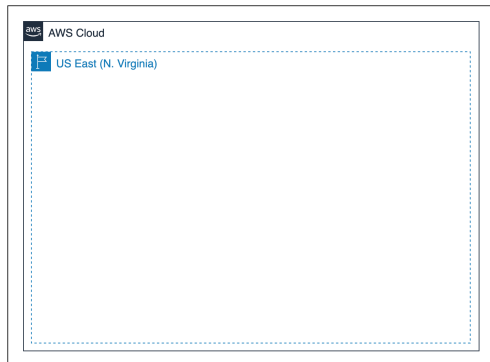
An understanding of EC2 instances

Working knowledge of key pairs in AWS

Working knowledge of security groups in AWS

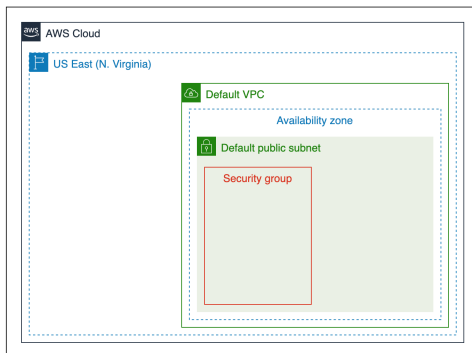
The ability to launch and manage new Amazon EC2 instances

Hands-on experience in deploying web applications on Amazon EC2 instances



Create a Security Group

Security groups are rules that manage the inbound and outbound traffic for an associated instance at the instance level. In other words, they can be considered a kind of firewall for the instance.



The architecture diagram

Let's create a security group before assigning it to our instance:

- On the AWS console, search for "EC2" and click the EC2 service from the search results. This takes us to the EC2 dashboard.
- In the sidebar, click the "Security Groups" option under the "Network & Security" section. A list of existing security groups appears.
- Click the "Create security group" button to start the creation process.

Follow these steps to configure and create the security group:

- **Basic details**
 - Enter `aws-bootstrap-security-group` as the security group name.
 - Enter `Security group for aws-bootstrap` as the description.
 - Select the default VPC under the "VPC" option, which associates the security group with your network.
- **Inbound rules**
 - Add the first inbound rule as follows:
 - Click the "Add rule" button under "Inbound rules."
 - Select `SSH` as the type, allowing SSH access to the instance.
 - Select `Anywhere-IPv4` as the source, permitting connections from any IPv4 address.
 - Add the second inbound rule as follows:
 - Click the "Add rule" button under "Inbound rules."
 - Select `HTTP` as the type, enabling web traffic on port 80.
 - Select `Anywhere-IPv4` as the source, allowing access from any IPv4 address.

- **Outbound rules**

- Keep the existing default rule as we do not need to change or add any outbound rules, which allows all outbound traffic by default.

- **Tags**

- Click the “Add new tag” button.
- Enter **Name** as the key and **aws-bootstrap-security-group** as the value to label the security group for easier identification.

- Once you’re done with the steps above, click the “Create security group” button to complete the process.

Note: The SSH connection will allow us to use “EC2 Instance Connect” on the AWS console, whereas the HTTP connection will allow us to connect to the instance using a browser.

Create a Key Pair

In this task, we’ll create a key pair that we will attach to the instance later and use to connect to it directly in a local environment.

Now, let’s create a key pair by following the steps below:

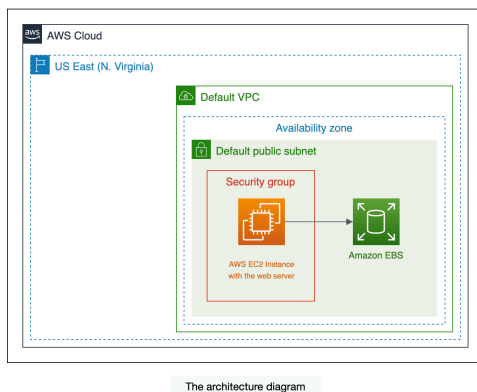
- In the sidebar, click the “Key Pairs” option under the “Network & Security” section. This displays a list of existing key pairs.
- Click the “Create key pair” button to start the creation process.
- Follow these steps:
 - Enter **aws-bootstrap-key-pair** as the key pair name, which will be used to identify your key pair.
 - Select **RSA** as the key pair type. RSA is a widely used encryption algorithm.
 - Select **.pem** as the private key file format, suitable for use with OpenSSH or PuTTY, respectively.
- Once you’ve filled out the details, click the “Create key pair” button to generate the key pair.

After we’ve successfully created our key pair, the **aws-bootstrap-key-pair.pem** file containing the key should be able to be downloaded. The created key pair should also appear in the list of key pairs.

Launch an Instance

In this task, we’ll create an instance to host a simple web application that we can connect with.

We’ll first configure the instance by specifying its AML, instance type, and storage, attaching the security group and key pair we previously created, and defining a bootstrap script to pre-launch the web application before finally launching it.



Step 1: Add instance name and tags

- Set **aws-bootstrap-instance** as the name of the instance.
- We can skip adding additional tags for now and proceed to the next step.

Step 2: Choose an Amazon Machine Image (AMI)

An **Amazon Machine Image (AMI)** is a preconfigured virtual machine image containing essential configuration data, such as the operating system type, web server, and software packages required to launch an EC2 instance.

For this lab, we will utilize one of the most popular AMIs provided and maintained by AWS to create the instance.

- In the “Quick Start” menu, select the “Amazon Linux” image.
- Select the **Amazon Linux 2023** AMI.
- Select the **64-bit (x86)** architecture.

Step 3: Choose an instance type

An **instance type** is a combination of various AWS resources like CPU, memory size, and storage that we want to attach to the instance according to our business needs.

For this lab, we’ll use the **t3.micro** instance type, which is normally used for running inexpensive and general-purpose tasks.

- Select the **t3.micro** type from the list.

Step 4: Select key pairs

- From the “Key pair name” drop-down menu, select the **aws-bootstrap-key-pair** option.

Step 5: Configure network settings

- Select the “Select existing security group” option.
- From the security groups drop-down menu, select the `aws-bootstrap-security-group` security group we previously created.

Step 6: Configure storage

- The 8 GiB **Amazon EBS gp3** storage should already be selected, which will be attached to the instance we’re about to launch. We don’t need to add any new storage for this instance or modify the existing one. Leave everything at its default setting and proceed to the next step.

Step 7: Configure advanced details

- Here, we only need to adjust the metadata and user data options:

Metadata

In this section, we will enable metadata access so that we will be able to fetch the information about an instance for configuring or management purposes when we’re connected to it.

- Select “Enabled” under “Metadata accessible.”
- Select “V1 and V2 (token optional)” under “Metadata version.”
- Select “Enable” under “Allow tags in metadata.”

User data

The “user data” section is the same as the bootstrapping process, which involves running a script on the instance on instance launch to install any required dependencies for the user on our server.

- Copy the following script and paste it in the “User data” textbox to launch a basic web server.

```
#!/bin/bash
sudo su
yum update -y
yum install -y httpd.x86_64
systemctl start httpd.service
systemctl enable httpd.service
echo "Hello World from $(hostname)" > /var/www/html/index.html
```

Step 8: Launch instance

- Review settings for the new instance before clicking the “Launch instance” button to launch the instance.

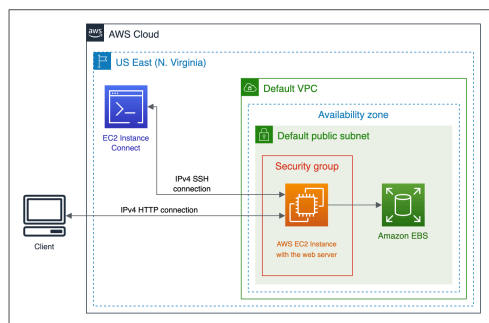
Finally, click “View all instances” to return to the “Instances” page. Within a few minutes, the status of our newly created instance should automatically change to “Running.”

Wait a minute or two before proceeding to the next task to allow the user data script to finish execution.

Test the Instance

In this task, we’ll test the EC2 instance we created by trying to reach the `httpd` server that was created using the user data script. We will first try to establish an HTTP connection to the instance’s public IP/DNS address. Then, we’ll establish an SSH connection through the AWS “EC2 Instance Connect” service. Finally, we’ll establish an SSH connection again on a Linux system using the `aws-bootstrap-key-pair` key pair.

By the end of this task, we would have built the AWS infrastructure represented below as an architecture diagram:



The architecture diagram

We will be following three approaches to carry out this test:

- Approach 1: Connecting to the deployed server on the EC2 instance through the instance’s public IP/DNS address on the browser.
 1. In the sidebar, click the “Instance” option under “Instance” section.
 2. Select the `aws-bootstrap-instance` instance we created from the list of instances.
 3. If it isn’t already open, select and open the bottom panel showing the instance metadata.
 4. Copy either the “Public IPv4 address” or “Public IPv4 DNS” and open it in a new tab, but make sure to use HTTP and not HTTPS in the URL. Otherwise the website won’t respond.
 5. If all goes well, we should be greeted with our custom `Hello World from $(hostname)` message on the website.
- Approach 2: Connecting to the EC2 instance through the AWS “EC2 Instance Connect” service.
 1. In the sidebar, click the “Instances” option under “Instances” section.
 2. Select the `aws-bootstrap-instance` instance we created from the list of instances.
 3. Next, click the “Connect” button.
 4. Upon being redirected, make sure the “EC2 Instance Connect” section is opened and click the “Connect” button to open up the EC2 terminal in a new tab.
 5. To fetch the instance’s metadata, use the `169.254.169.254` IP address, a link-local IP address that AWS uses to provide a metadata service to EC2 instances. In response, it’ll list the metadata categories that can be fetched.

<http://169.254.169.254/latest/meta-data/>

For example, to fetch the hostname of the instance, use the command below. Compare it with the hostname displayed on the web server we launched.

```
curl http://169.254.169.254/latest/meta-data/hostname
```

Now, type in the following command to ping the server running on the instance and see if it responds with our custom `Hello World from $(hostname)` message.

```
curl localhost:80
```

If all goes well, we should be greeted with our custom hello message in the terminal.

- Approach 3: Connecting to the EC2 instance through a Linux environment using key pairs.
 1. First, open the downloaded `aws-bootstrap-key-pair` in an IDE like VS Code or a text editor on your local machine, and then copy the contents of the file and paste it inside the `aws-bootstrap-key-pair.pem` file in the code widget below.
 2. Click the “Run” button of the code widget to initiate the widget terminal, which has the Linux Ubuntu 20.04 environment set up.
 3. Now, we need to add the read permission to the key pair for which we use `chmod`. Copy, paste, and enter the command in the terminal of the code widget below.

```
chmod 400 /usercode/aws-bootstrap-key-pair.pem
```

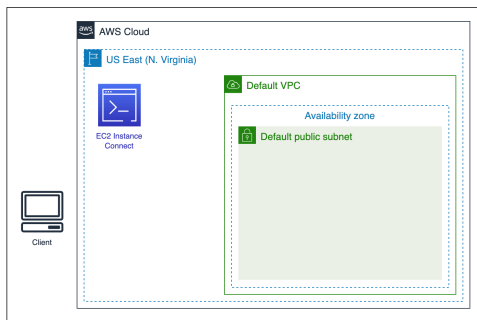
1. Now, to connect to the instance through the widget’s terminal, we need to run the SSH command below. Make sure to replace `<INSTANCE_DNS_ADDRESS>` with the public DNS address of our instance.

```
ssh -i /path to the AWS keys/aws-bootstrap-key-pair.pem ec2-user@<INSTANCE_DNS_ADDRESS>
```

2. A prompt should appear asking whether we want to proceed with connecting. Type in `yes` to continue. If the credentials are valid, the EC2 instance terminal similar to the one in the “EC2 Instance Connect” service will appear.
3. To test further, repeat steps 5–8, which we’ve previously written for the AWS “EC2 Instance Connect” service in approach 2.

Clean Up

Before concluding this lab, we need to clean up any resources we created for this lab. After the completion of this task, the architecture diagram of our AWS infrastructure will be as follows:



The architecture diagram

Terminate the instance

- Close the “EC2 Instance Connect” shell and return to the EC2 dashboard.
- Open up the sidebar and click the “Instances” option under “Instances” section.
- Select the `aws-bootstrap-instance` instance from the list.
- Click the “Instance state” drop-down button and then click the “Terminate instance” option.
- Click the “Terminate” button.

Delete the key pair

- Open up the sidebar and click the “Key Pairs” option under “Network & Security.”
- Select the `aws-bootstrap-key-pair` key pair from the list.
- Click the “Action” drop-down button and then click the “Delete” option.
- Enter `Delete` in the pop-up prompt and click the “Delete” button.

Delete the security group

- We need to ensure that the `aws-bootstrap-instance` instance is terminated. Otherwise, we won’t be able to delete the security group.
- Open up the sidebar and click the “Security Groups” option under “Network & Security.”
- Select the `aws-bootstrap-security-group` security group from the list if it isn’t already selected.
- Click the “Action” drop-down button and then click the “Delete security groups” option.
- In the pop-up prompt, click the “Delete” button.

Convert PEM file to PPK for Putty

- Open PuTTYgen on your Windows machine.
- Click the Load button.
- In the file explorer popup, change the file extension dropdown (bottom-right) from "PuTTY Private Key Files (.ppk)" to ****All Files (*.*)****.
- Select your downloaded AWS .pem key file and click Open.
- Click OK on the successful import notification.
- Click Save private key.
- Click Yes when warned about saving without a passphrase (or specify one for extra security).
- Name your file (e.g., ec2-key.ppk) and save it to a secure local folder. [[1](#) [2](#) [3](#) [4](#) [5](#) [6](#)]